Clase 3: Procesos - El Corazón del Sistema Operativo

Introducción: ¿Qué es un proceso?

Cuando ejecutás un programa, el sistema operativo crea algo llamado **proceso**. Pero un proceso no es simplemente "un programa corriendo" - es mucho más que eso. Un proceso es una instancia de un programa en ejecución junto con todo su contexto: la memoria que usa, los archivos que tiene abiertos, su estado de ejecución, y su relación con otros procesos.

Pensalo así: si el programa es la receta, el proceso es la acción de cocinar. Podés tener la misma receta (programa) pero cocinarla múltiples veces simultáneamente (múltiples procesos). Cada vez que cocinás, tenés tus propios ingredientes (memoria), tus propias ollas (file descriptors), y tu propio progreso (program counter).

Esta distinción es fundamental para entender sistemas operativos. Un programa es pasivo - bytes almacenados en disco. Un proceso es activo - una entidad viva que el kernel gestiona, le asigna CPU, y eventualmente termina.

El modelo de procesos en UNIX

Una jerarquía familiar

UNIX organiza los procesos en una estructura de árbol. Cada proceso

(excepto el primero) tiene un padre que lo creó. Este modelo jerárquico no es un accidente de diseño - refleja cómo los programas naturalmente delegan trabajo a subprocesos.

Cuando arranca tu sistema Linux, el kernel crea el proceso con PID 1, tradicionalmente llamado `init` (hoy es `systemd` en la mayoría de distribuciones). Este proceso es el ancestro de todos los demás. Cada vez que abrís una terminal, `systemd` (o algún descendiente suyo) crea un proceso shell. Cuando en ese shell escribís un comando, el shell crea un proceso hijo para ejecutarlo.

```
# Ver el árbol de procesos
pstree -p

# Salida típica (simplificada):
# systemd(1)─┬─sshd(1234)───sshd(5678)───bash(5680)───pyt
#            ├─dockerd(2345)───containerd(2346)
#            └─nginx(3456)─┬─nginx(3457)
#                          └─nginx(3458)
```

Esta jerarquía tiene consecuencias importantes. Cuando un proceso padre termina, sus hijos quedan "huérfanos" y son adoptados por init/systemd. Cuando un hijo termina pero su padre no recoge su estado de salida, el hijo queda como "zombie" - un proceso muerto que aún ocupa una entrada en la tabla de procesos.

Anatomía de un proceso

Cada proceso en Linux tiene varios componentes esenciales:

PID (Process ID): Un número único que identifica al proceso. Los PIDs se asignan secuencialmente y eventualmente se reciclan cuando se agotan.

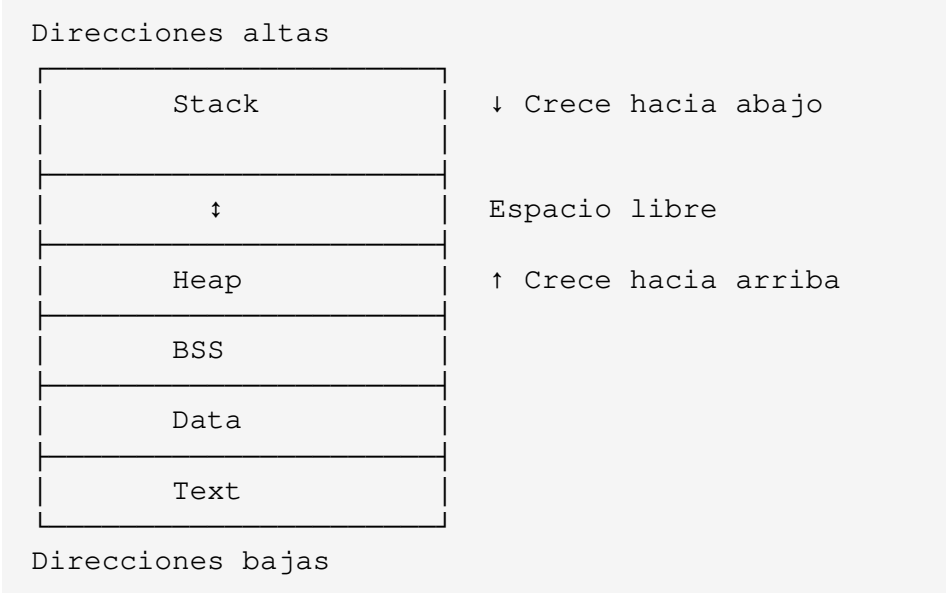
PPID (Parent Process ID): El PID del proceso padre. Esto forma la cadena jerárquica.

Estado: Running (ejecutándose o listo para ejecutar), Sleeping (esperando algo), Stopped (detenido por una señal), Zombie (terminado

pero no recogido por el padre).

Memoria virtual: Cada proceso tiene su propio espacio de direcciones virtuales, completamente aislado de otros procesos. El kernel y la MMU (Memory Management Unit) del procesador se encargan de traducir las direcciones virtuales que usa el programa a direcciones físicas reales en la RAM. Esto significa que dos procesos pueden usar la "misma" dirección de memoria virtual sin conflicto — cada uno apunta a una ubicación física distinta.

El espacio de memoria de un proceso se organiza en segmentos con propósitos específicos:



- **Text segment (código):** Contiene las instrucciones del programa compilado. Es de **solo lectura** — si un proceso intenta escribir acá, el kernel lo mata con una señal SIGSEGV (segmentation fault). Es compartible: si corrés 10 veces el mismo programa, las 10 instancias pueden compartir la misma copia del text segment en memoria física.
- **Data segment (datos inicializados):** Almacena las variables globales y estáticas que tienen un valor inicial asignado en el código. Por ejemplo, si en C escribís `int contador = 42;`,

esa variable vive acá. El valor inicial se copia del ejecutable al cargar el programa.

- **BSS (Block Started by Symbol):** Almacena las variables globales y estáticas que **no** fueron inicializadas explícitamente (o se inicializaron a cero). El nombre viene del ensamblador de los años '50 y quedó por tradición. La diferencia con Data es que BSS no ocupa espacio en el archivo ejecutable — el kernel simplemente reserva la memoria y la llena de ceros al cargar el proceso. Si declarás `int tabla[1000000];` sin inicializar, no vas a tener un ejecutable de 4MB; el SO sabe que necesita reservar ese espacio y ponerlo en cero.

- **Heap (montículo):** Memoria dinámica que el programa solicita en tiempo de ejecución. En C se obtiene con `malloc()`/`free()`, en Python el intérprete la gestiona internamente para crear objetos. Crece hacia direcciones altas. Si se queda sin espacio, el proceso puede pedir más al kernel (vía `brk()` o `mmap()`).

- **Stack (pila):** Almacena las variables locales de funciones, los argumentos pasados a funciones, y las direcciones de retorno (para saber adónde volver cuando una función termina). Crece hacia direcciones bajas — es decir, en dirección opuesta al heap. Cada vez que llamás a una función se apila un **stack frame**; cuando la función retorna, ese frame se desapila. Si se crece demasiado (por ejemplo, recursión infinita), el proceso recibe un stack overflow.

El hecho de que heap y stack crezcan en direcciones opuestas es un diseño deliberado: aprovechan el mismo espacio libre desde extremos opuestos.

Podés ver el mapa de memoria de cualquier proceso en Linux:

```
# Ver el layout de memoria de un proceso
cat /proc/<pid>/maps

# Ejemplo práctico en Docker:
python3 -c "
import os
```

```
print(f'Mi PID es {os.getpid()}')
input('Presioná Enter para salir...')
" &
cat /proc/$!/maps
```

File descriptors: Una tabla de archivos abiertos. Por defecto, cada proceso hereda tres: stdin (0), stdout (1), stderr (2).

Credenciales: UID y GID que determinan los permisos del proceso.

Creando procesos: fork()

El concepto fundamental

La llamada al sistema `fork()` es una de las ideas más elegantes (y a veces confusas) en UNIX. Cuando un proceso llama a `fork()`, el kernel crea una copia casi exacta del proceso. Después del fork, tenés dos procesos ejecutando el mismo código desde el mismo punto.

Lo brillante de fork es su simplicidad conceptual: no necesitás especificar qué ejecutar o cómo configurar el nuevo proceso. Simplemente decís "duplicame" y obtenés una copia.

```
import os

print(f"Antes del fork, soy el proceso {os.getpid()}")

pid = os.fork()

if pid == 0:
    # Este código lo ejecuta el HIJO
    print(f"Soy el hijo, mi PID es {os.getpid()}, mi padr
else:
    # Este código lo ejecuta el PADRE
    print(f"Soy el padre (PID {os.getpid()}), creé al hij
```

```
print(f"Este mensaje lo imprimen AMBOS procesos (PID {os.
```

Salida típica:

```
Antes del fork, soy el proceso 1234
Soy el padre (PID 1234), creé al hijo con PID 1235
Este mensaje lo imprimen AMBOS procesos (PID 1234)
Soy el hijo, mi PID es 1235, mi padre es 1234
Este mensaje lo imprimen AMBOS procesos (PID 1235)
```

¿Cómo distinguir padre de hijo?

El valor de retorno de `fork()` es la clave:

- En el **proceso padre**: fork() retorna el PID del hijo (número positivo)
- En el **proceso hijo**: fork() retorna 0
- Si hay **error**: fork() retorna -1 (solo en el padre, el hijo nunca se crea)

Este diseño permite que el mismo código tome caminos diferentes según si es padre o hijo:

```
import os

pid = os.fork()

if pid < 0:
    print("Error: no se pudo crear el proceso")
elif pid == 0:
    # Rama del hijo
    print("Soy el hijo, voy a hacer trabajo específico de
    # El hijo típicamente hace algo diferente y luego ter
    os._exit(0)  # Terminar el hijo
else:
    # Rama del padre
    print(f"Soy el padre, mi hijo es {pid}")
```

```
    # El padre típicamente espera al hijo o continúa con
```

Copy-on-Write: la eficiencia detrás de fork

Si fork realmente copiara toda la memoria del proceso padre, sería tremendamente ineficiente. Un proceso puede tener gigabytes de memoria - ¿copiar todo eso para cada fork?

Linux usa una técnica llamada **Copy-on-Write (COW)**. Inmediatamente después del fork, padre e hijo comparten las mismas páginas de memoria física, marcadas como solo lectura. Solo cuando alguno de los dos intenta escribir en una página, el kernel crea una copia de esa página específica.

Esto significa que fork es muy rápido - solo necesita copiar las estructuras de control del proceso, no la memoria en sí. Y si el hijo inmediatamente hace exec (como veremos), las páginas compartidas nunca necesitan copiarse.

Reemplazando el programa: exec()

Fork crea, exec transforma

Fork crea un nuevo proceso, pero ese proceso ejecuta el mismo programa que el padre. ¿Qué pasa si queremos que el hijo ejecute un programa diferente? Para eso está la familia de funciones `exec`.

Exec no crea un nuevo proceso - reemplaza completamente el programa del proceso actual. La memoria se limpia, se carga el nuevo ejecutable, y la ejecución comienza desde el inicio del nuevo programa. El PID permanece igual, pero todo lo demás cambia.

```
import os
```

```
pid = os.fork()

if pid == 0:
    # El hijo va a transformarse en 'ls'
    print(f"Hijo (PID {os.getpid()}): voy a convertirme e
    os.execlp("ls", "ls", "-la", "/tmp")
    # Si llegamos aquí, exec falló
    print("Error: exec falló")
    os._exit(1)
else:
    print(f"Padre: esperando que el hijo {pid} termine...
    os.wait()
    print("Padre: el hijo terminó")
```

La familia exec

Python (a través del módulo `os`) expone varias variantes de exec:

```
# Las variantes difieren en cómo especifican el programa

# execl: argumentos como lista de parámetros
os.execl("/bin/ls", "ls", "-l", "/home")

# execlp: busca en PATH
os.execlp("ls", "ls", "-l", "/home")

# execle: permite especificar environment
os.execle("/bin/ls", "ls", "-l", env={"PATH": "/bin"})

# execv: argumentos como lista/tupla
os.execv("/bin/ls", ["ls", "-l", "/home"])

# execvp: busca en PATH + argumentos como lista
os.execvp("ls", ["ls", "-l", "/home"])

# execve: la más fundamental - path, args, env
os.execve("/bin/ls", ["ls", "-l"], {"PATH": "/bin"})
```

La convención de nombres:

- **l** (list): argumentos como parámetros separados
- **v** (vector): argumentos como lista
- **p** (path): busca el ejecutable en PATH
- **e** (environment): permite especificar variables de entorno

El patrón fork-exec

La combinación fork + exec es el patrón fundamental para ejecutar programas en UNIX:

```
import os
import sys

def ejecutar_comando(comando, args):
    """Ejecuta un comando externo y espera a que termine.
    pid = os.fork()

    if pid == 0:
        # Hijo: ejecutar el comando
        try:
            os.execvp(comando, [comando] + args)
        except OSError as e:
            print(f"Error ejecutando {comando}: {e}", fil
            os._exit(1)
    else:
        # Padre: esperar al hijo
        _, status = os.waitpid(pid, 0)
        return os.WEXITSTATUS(status)

# Uso
codigo_salida = ejecutar_comando("ls", ["-la", "/tmp"])
print(f"ls terminó con código {codigo_salida}")
```

Esperando procesos: wait()

El problema de los zombies

Cuando un proceso termina, no desaparece inmediatamente. El kernel mantiene información sobre cómo terminó (código de salida, si fue por una señal, etc.) hasta que el padre la recoja. Un proceso en este estado se llama **zombie**.

Los zombies no consumen CPU ni memoria significativa, pero ocupan una entrada en la tabla de procesos del kernel. Si un programa crea muchos hijos sin recogerlos, puede agotar la tabla de procesos.

```
import os
import time

# Crear un zombie
pid = os.fork()
if pid == 0:
    # El hijo termina inmediatamente
    print("Hijo: terminando")
    os._exit(0)
else:
    # El padre NO hace wait - el hijo queda zombie
    print(f"Padre: creé hijo {pid}, pero no voy a esperar
    print("Ejecutá 'ps aux | grep Z' en otra terminal par
    time.sleep(30)  # Mantener el padre vivo
```

Usando wait

La solución es que el padre llame a `wait()` o `waitpid()`:

```
import os

pid = os.fork()

if pid == 0:
    print("Hijo: trabajando...")
    os._exit(42)  # Terminar con código 42
```

```
else:
    print(f"Padre: esperando al hijo {pid}")

    # wait() bloquea hasta que ALGÚN hijo termine
    hijo_terminado, status = os.wait()

    # Extraer el código de salida
    if os.WIFEXITED(status):
        codigo = os.WEXITSTATUS(status)
        print(f"Hijo {hijo_terminado} terminó normalmente
    elif os.WIFSIGNALED(status):
        señal = os.WTERMSIG(status)
        print(f"Hijo {hijo_terminado} terminado por señal
```

waitpid: más control

`waitpid` permite esperar a un hijo específico y tiene opciones adicionales:

```
import os

# Crear varios hijos
hijos = []
for i in range(3):
    pid = os.fork()
    if pid == 0:
        os._exit(i)  # Cada hijo sale con código diferent
    hijos.append(pid)

# Esperar a cada hijo específicamente
for pid in hijos:
    _, status = os.waitpid(pid, 0)  # 0 = bloquear
    print(f"Hijo {pid} terminó con código {os.WEXITSTATUS

# Con WNOHANG: no bloquear si el hijo no terminó
pid, status = os.waitpid(-1, os.WNOHANG)
if pid == 0:
    print("Ningún hijo terminó todavía")
```

Procesos en práctica: el shell

Para entender cómo funcionan los procesos, pensemos en cómo funciona un shell simplificado:

```
import os
import sys

def shell_simple():
    while True:
        # 1. Mostrar prompt y leer comando
        try:
            linea = input("$ ")
        except EOFError:
            break

        if not linea.strip():
            continue

        partes = linea.strip().split()
        comando = partes[0]
        args = partes[1:]

        # Comandos internos (no hacen fork)
        if comando == "exit":
            break
        if comando == "cd":
            try:
                os.chdir(args[0] if args else os.environ[
            except OSError as e:
                print(f"cd: {e}")
            continue

        # Comandos externos: fork + exec
        pid = os.fork()
```

```
        if pid == 0:
            # Hijo: ejecutar el comando
            try:
                os.execvp(comando, [comando] + args)
            except OSError as e:
                print(f"{comando}: {e}")
                os._exit(1)
        else:
            # Padre: esperar al hijo
            _, status = os.wait()
            if os.WIFEXITED(status):
                codigo = os.WEXITSTATUS(status)
                if codigo != 0:
                    print(f"[Salió con código {codigo}]")

if __name__ == "__main__":
    shell_simple()
```

Este shell minimalista demuestra los conceptos fundamentales:

1. El shell es un proceso que corre en un loop
2. Para comandos externos, hace fork para crear un hijo
3. El hijo hace exec para transformarse en el comando
4. El padre hace wait para esperar que el hijo termine
5. Comandos como `cd` deben ser internos (¿por qué?)

Comunicación entre padre e hijo

Herencia en fork

El hijo hereda muchas cosas del padre:

- Espacio de memoria (como copia, con COW)
- File descriptors abiertos
- Variables de entorno

- Directorio de trabajo actual
- Máscara de señales
- UID, GID, y grupos suplementarios

El hijo NO hereda:

- El PID (obvio - tiene uno nuevo)
- Locks de archivos
- Timers y alarmas pendientes
- Señales pendientes

Comunicación básica: código de salida

La forma más simple de comunicación es el código de salida:

```
import os
import sys

def procesar_archivo(archivo):
    """El hijo procesa un archivo y reporta éxito/fallo v
    pid = os.fork()

    if pid == 0:
        try:
            with open(archivo) as f:
                lineas = len(f.readlines())
            print(f"{archivo}: {lineas} líneas")
            os._exit(0)  # Éxito
        except Exception as e:
            print(f"Error procesando {archivo}: {e}")
            os._exit(1)  # Fallo
    else:
        _, status = os.wait()
        return os.WEXITSTATUS(status) == 0

# Procesar varios archivos
archivos = ["/etc/passwd", "/etc/no_existe", "/etc/hosts"
for archivo in archivos:
    if procesar_archivo(archivo):
```

```
        print(f"  ✓ {archivo} procesado correctamente")
    else:
        print(f"  ✗ {archivo} falló")
```

Variables de entorno

El padre puede pasar información al hijo vía variables de entorno:

```
import os

# Configurar antes del fork
os.environ["MODO"] = "produccion"
os.environ["DEBUG"] = "0"

pid = os.fork()

if pid == 0:
    # El hijo ve las variables
    print(f"Hijo: MODO={os.environ.get('MODO')}")
    os._exit(0)
else:
    os.wait()
```

Ejercicio guiado: monitor de procesos

Vamos a construir un pequeño monitor que ejecute varios comandos en paralelo:

```
#!/usr/bin/env python3
"""
Monitor que ejecuta comandos en paralelo y reporta result
"""
import os
import sys
```

```
import time

def ejecutar_paralelo(comandos):
    """
    Ejecuta una lista de comandos en paralelo.
    Retorna dict con {pid: (comando, codigo_salida)}
    """
    procesos = {}  # pid -> comando

    # Crear un hijo por cada comando
    for cmd in comandos:
        pid = os.fork()

        if pid == 0:
            # Hijo: ejecutar comando
            partes = cmd.split()
            try:
                os.execvp(partes[0], partes)
            except OSError as e:
                print(f"Error ejecutando '{cmd}': {e}", f
                os._exit(127)
        else:
            # Padre: registrar el hijo
            procesos[pid] = cmd
            print(f"[{pid}] Iniciado: {cmd}")

    # Esperar a todos los hijos
    resultados = {}
    while procesos:
        pid, status = os.wait()
        cmd = procesos.pop(pid)
        codigo = os.WEXITSTATUS(status) if os.WIFEXITED(s
        resultados[pid] = (cmd, codigo)
        print(f"[{pid}] Terminado: {cmd} (código {codigo}

    return resultados

if __name__ == "__main__":
    comandos = [
        "sleep 2",
```

```
        "ls /tmp",
        "echo hola mundo",
        "sleep 1",
    ]

    print("=== Ejecutando comandos en paralelo ===")
    inicio = time.time()
    resultados = ejecutar_paralelo(comandos)
    duracion = time.time() - inicio

    print(f"\n=== Resumen (duración total: {duracion:.1f}
    exitos = sum(1 for _, (_, codigo) in resultados.items
    print(f"Exitosos: {exitos}/{len(resultados)}")
```

Conceptos clave para recordar

1. **Proceso vs Programa:** El programa es código en disco, el proceso es ejecución activa con su contexto.
2. **fork() duplica:** Crea una copia del proceso. El padre recibe el PID del hijo, el hijo recibe 0.
3. **exec() transforma:** Reemplaza el programa del proceso actual. El PID no cambia.
4. **wait() recoge:** El padre debe esperar a los hijos para evitar zombies y obtener su estado de salida.
5. **El patrón fork-exec:** La forma estándar de ejecutar programas externos en UNIX.
6. **Copy-on-Write:** Fork es eficiente porque no copia memoria hasta que es necesario.
7. **Jerarquía de procesos:** Todo proceso tiene un padre. init/systemd es el ancestro de todos.

Preparación para la próxima clase

En la clase 4 veremos cómo los procesos se comunican entre sí usando pipes, el mecanismo fundamental de UNIX para conectar la salida de un proceso con la entrada de otro.

Pensá en esto: cuando escribís `ls | grep txt` en el shell, ¿cómo conecta el shell la salida de `ls` con la entrada de `grep`? La respuesta involucra fork, exec, y pipes - todo trabajando juntos.

Computación II - 2026 - Clase 3